

NAME: KRISHITHA KANKANALA

CLASS: 5D

SRN: PES2UG24CS234

GITHUB: https://github.com/krista9669/SE_LAB1

Database Query Performance Profiler

Problem Statement #44 — Developer Tools & IT Operations

1. Requirements Table

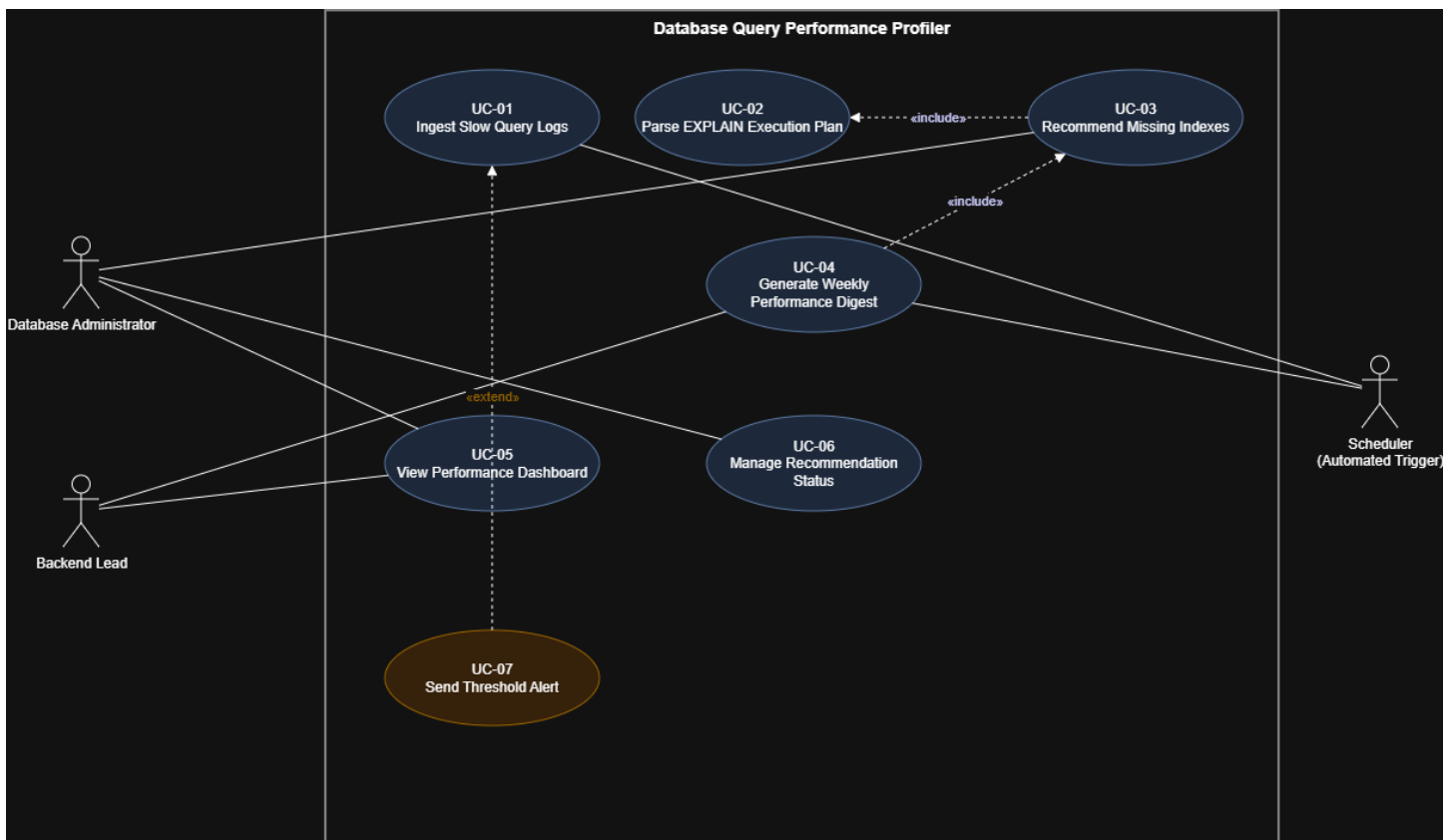
Functional Requirements

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall parse Postgres/MySQL EXPLAIN plans, highlight sequential table scans on large tables, and recommend optimal index column definitions.	High	Pass: Missing composite index identified on filtered columns. Fail: Query plan with sequential scan marked optimized.	Core capability that enables all downstream index recommendations.
FR-002	Functional	The system shall ingest slow query logs from Postgres and MySQL log files or streaming sources on a continuous basis.	High	Pass: A slow query written to the source log appears in the system's log store within 30 seconds. Fail: Entry missing or delayed beyond 60 seconds.	Ingestion is the entry point for all analysis; without it no query can be profiled.
FR-003	Functional	The system shall generate a weekly performance optimization digest summarizing the top 10 slowest queries, recommended indexes, and estimated performance gain for each.	High	Pass: A digest is generated every Monday 06:00 IST containing exactly 10 ranked queries with recommendations. Fail: Digest missing, incomplete, or unranked.	Gives the DBA and Backend Lead a scheduled, digestible summary instead of manual log review.
FR-004	Functional	The system shall allow the Database Administrator to mark an index recommendation as Applied, Dismissed, or Pending and persist that status.	Medium	Pass: Status change is saved and reflected in the recommendation list on next page load. Fail: Status reverts or is not saved.	Prevents duplicate recommendations and tracks remediation progress over time.
FR-005	Functional	The system shall send an alert notification (email) to the Backend Lead when a single query's execution time exceeds a configurable threshold (default 5000 ms).	Medium	Pass: An alert is received within 2 minutes of a query breaching the threshold. Fail: No alert sent, or sent for a query below threshold.	Enables proactive response to acute performance regressions rather than waiting for the weekly digest.

Non-Functional Requirements

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Nonfunctional (Performance & Security)	Slow query log ingestion must handle up to 5,000 log records per minute without dropping entries or slowing the host DB.	High	Pass: Benchmarking tests confirm target latency and security standards under simulated peak load.	Guarantees the profiler scales to production-level traffic without impacting the monitored database.
NFR-002	Nonfunctional (Security)	The system shall encrypt slow query log data at rest and enforce role-based access control so that only authenticated Database Administrator and Backend Lead accounts can view query text and recommendations.	High	Pass: Unauthorized/unauthenticated requests to the API or dashboard are rejected with HTTP 401/403; stored log data is AES-256 encrypted at rest. Fail: Any unauthenticated access succeeds or data is stored in plaintext.	Query logs can contain sensitive data (e.g. literal values in WHERE clauses), so access must be restricted and data protected.

2. UML Use-Case Diagram



Actors: Database Administrator, Backend Lead, Scheduler (Automated Trigger).

«include»: UC-03 includes UC-02; UC-04 includes UC-03.

«extend»: UC-07 extends UC-01.

3. Use-Case Flow

Use Case	Recommend Missing Indexes
Actor	Database Administrator (primary), with Scheduler (automated trigger) as a secondary actor
Postconditions	A ranked list of index recommendations is generated, persisted with status "Pending", and made visible to the Database Administrator on the dashboard.
Main Flow	1. Database Administrator selects a time window and requests an index-recommendation analysis (or the Scheduler triggers the same analysis automatically on its configured interval). 2. System retrieves all slow-query log entries ingested within the selected window. 3. System parses the EXPLAIN plan attached to each query. 4. System flags queries whose plan contains a sequential scan on a table exceeding the configured row-count threshold. 5. System inspects the filter and join columns referenced by each flagged query. 6. System generates a candidate index definition (single-column or composite) for each flagged query. 7. System estimates the performance improvement each candidate index would provide and ranks all candidates accordingly. 8. System stores the ranked recommendation list with status "Pending" and displays it to the Database Administrator. 9. Use case ends successfully.
Alternate Flow	4a. No Sequential Scans Found 4a1. System completes analysis of the window and finds no query whose plan contains a sequential scan above the configured threshold. 4a2. System records "No recommendations for this window" and displays this status to the Database Administrator instead of a recommendation list. 4a3. Use case ends without generating recommendations.